

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Бинарное дерево поиска»

Выполнили студенты группы 25BBB2

Лампетов А.С.

Ковтун А.А.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Деев М.В.

Пенза 2026

Тема: Бинарное дерево поиска

Цель работы:

изучение бинарных деревьев поиска и реализация на языке Go основных операций над ними: добавления элемента, вывода дерева, поиска значения и подсчёта числа вхождений элемента; оценка сложности поиска.

Лабораторное задание:

1. Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.
2. Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.
3. * Изменить функцию добавления элементов для исключения добавления одинаковых символов.
4. * Оценить сложность процедуры поиска по значению в бинарном дереве.

Ход работы:

Бинарное дерево поиска реализовано на основе структуры `Node`, которая содержит целочисленное информационное поле `data` и два указателя: `left` и `right` — на левого и правого потомков. Указатель на корень дерева хранится в переменной `root`; пустому дереву соответствует значение `nil`. Как и в исходном примере из задания, значения большие, чем значение в узле, помещаются в левое поддерево, а меньшие — в правое.

Функция добавления `CreateTree` рекурсивно спускается по дереву до первого свободного места и создаёт там новый узел. Функция возвращает корень текущего поддерева, поэтому присваивание `root.left = CreateTree(root.left, data)` связывает созданный узел с родителем без передачи дополнительных указателей. В отличие от исходного варианта на языке C, в Go не требуется вручную выделять и проверять память: узел создаётся

выражением `&Node{data: data}`, а освобождением памяти занимается сборщик мусора.

Для выполнения пункта 3 задания в функцию добавлена проверка `data == root.data`: если значение уже есть в дереве, функция возвращает поддереву без изменений и дубликат не создаётся. Дополнительно перед вызовом `CreateTree` в главном меню выполняется поиск элемента, что позволяет сообщить пользователю о том, что элемент уже существует.

Структура узла и функция добавления элементов:

```
1 type Node struct {
2     data int
3     left *Node
4     right *Node
5 }
6
7 func CreateTree(root *Node, data int) *Node {
8     if root == nil {
9         return &Node{data: data}
10    }
11
12    if data == root.data { // исключаем одинаковые значения
13        return root
14    } else if data > root.data {
15        root.left = CreateTree(root.left, data)
16    } else {
17        root.right = CreateTree(root.right, data)
18    }
19
20    return root
21 }
```

Рисунок 1 — Структура узла и функция добавления элемента

Функция поиска `search` реализует алгоритм спуска по дереву. Значение в текущем узле сравнивается с искомым: если они равны, возвращается указатель на найденный узел; если искомое значение больше — поиск продолжается в левом поддереве, если меньше — в правом. Достижение пустого указателя (`nil`) означает, что элемента в дереве нет, и функция возвращает `nil`.

```

1 func search(r *Node, data int) *Node {
2     if r == nil {
3         return nil
4     }
5
6     if r.data == data {
7         return r
8     }
9
10    if data > r.data {
11        return search(r.left, data)
12    } else {
13        return search(r.right, data)
14    }
15 }

```

Рисунок 2 — Функция поиска значения

Функция `countOccurrences` возвращает число вхождений заданного значения. Если значение в текущем узле совпало с искомым, к единице прибавляется результат обхода обоих поддеревьев, так как в общем случае равные значения могут находиться по обе стороны от найденного узла; в остальных случаях спуск выполняется так же, как при поиске. Поскольку функция добавления теперь не создаёт дубликатов, результат подсчёта в построенном дереве равен 0 или 1. Функция при этом остаётся корректной и для деревьев, в которых одинаковые значения допускаются.

```

1 func countOccurrences(r *Node, data int) int {
2     if r == nil {
3         return 0
4     }
5
6     if r.data == data {
7         return 1 + countOccurrences(r.left, data) +
8             countOccurrences(r.right, data)
9     }
10
11    if data > r.data {
12        return countOccurrences(r.left, data)
13    } else {
14        return countOccurrences(r.right, data)
15    }
16 }

```

Рисунок 3 — Функция подсчёта числа вхождений

Вывод дерева выполняет функция `printTree`. Она обходит дерево в порядке «правое поддерево — узел — левое поддерево» и печатает каждый

узел с отступом, пропорциональным его глубине. В результате дерево выводится повёрнутым на 90 градусов: корень находится слева, а поддерево с меньшими значениями оказывается сверху, с большими — снизу.

Для удобной проверки работы всех функций написана программа с текстовым меню, которая позволяет добавлять элемент, искать элемент, выводить дерево и подсчитывать число вхождений. Перед каждым действием экран очищается функцией `clearConsole`, которая вызывает команду `cls` в Windows и `clear` в остальных операционных системах.

Оценка сложности поиска:

При поиске алгоритм на каждом шаге сравнивает искомое значение с текущим узлом и переходит ровно в одно поддерево, то есть движется по одному пути от корня вниз. Число операций сравнения поэтому не превышает высоты дерева h , и сложность поиска составляет $O(h)$. Объём дополнительной памяти при рекурсивной реализации также равен $O(h)$ за счёт стека вызовов.

В лучшем случае искомый элемент находится в корне, и поиск выполняется за $O(1)$. Если дерево сбалансировано, высота равна примерно $\log_2 n$, рекуррентное соотношение имеет вид $T(n) = T(n/2) + O(1)$, а сложность поиска равна $O(\log n)$. Например, для $n = 1\,000\,000$ элементов потребуется не более 20 сравнений. Для дерева, построенного из случайной последовательности, средняя высота также пропорциональна $\log n$. В худшем случае элементы добавлялись в отсортированном порядке, дерево вырождается в цепочку высотой n , $T(n) = T(n - 1) + O(1)$, и сложность поиска увеличивается до $O(n)$, как при последовательном просмотре списка. Сложность операции подсчёта вхождений в дереве без дубликатов такая же, как у поиска, — $O(h)$.

Результаты работы программы:

Ниже приведён результат добавления элемента: при повторной попытке добавить уже существующее значение дерево не изменяется, а пользователю выводится сообщение о том, что элемент уже существует.

```
Введите число для добавления: 50
Элемент 50 добавлен.
```

```
Введите число для добавления: 50
Элемент 50 уже существует.
```

Рисунок 4 — Добавление элемента и попытка добавить дубликат

В дерево последовательно были добавлены значения 50, 30, 70, 20, 40, 60 и 80. Так как большие значения размещаются в левом поддереве, в выводе корень 50 находится в середине, сверху расположено поддерево с меньшими значениями (30, 20, 40), снизу — с большими (70, 60, 80).

```
Дерево :
```

```
  20
```

```
  30
```

```
  40
```

```
 50
```

```
  60
```

```
  70
```

```
  80
```

Рисунок 5 — Вывод бинарного дерева поиска

Результаты поиска значения в построенном дереве: существующий элемент найден, отсутствующий — не найден.

```
Введите число для поиска: 60
Элемент 60 найден в дереве.
```

```
Введите число для поиска: 55
Элемент 55 не найден в дереве.
```

Рисунок 6 — Результаты поиска элемента в дереве

Результаты подсчёта числа вхождений: для элемента, присутствующего в дереве, получено значение 1, для отсутствующего — 0.

```
Введите число: 70
Количество вхождений элемента 70: 1

Введите число: 15
Количество вхождений элемента 15: 0
```

Рисунок 7 — Результаты подсчёта числа вхождений элемента

Во всех случаях результаты, выданные программой, совпали с ожидаемыми.

Вывод:

В ходе выполнения лабораторной работы были изучены бинарные деревья поиска и на языке Go реализованы основные операции над ними: добавление элемента, вывод дерева, поиск значения и подсчёт числа вхождений. Функция добавления была изменена так, чтобы одинаковые значения в дерево не попадали. Анализ показал, что сложность поиска в бинарном дереве равна $O(h)$, где h — высота дерева: для сбалансированного дерева это $O(\log n)$, а для вырожденного — $O(n)$. Результаты работы программы совпали с ожидаемыми, что подтверждает корректность реализации.